

Identificadores Estáveis Universais: Pure NanoID 14c

Eliminação de Tabelas YAML, Domínio Encurtador aresta.cc e Física do QR Code na Rocha

Projeto: Aresta ClimB • Repositório: [aresta_db](#) • Autor: Engenharia e Arquitetura Aresta • Data: Outubro de 2026 • Versão: 2.0 (Pure NanoID 14c)

1. Resumo Executivo: A Racionalidade do Pure NanoID 14c

Esta especificação consolida a decisão arquitetural definitiva para identificadores no ecossistema Aresta ClimB: a adoção universal de **Pure NanoID com 14 caracteres Base62 contínuo** (`[0-9a-zA-Z]`), ex: `x8siJek3FiG3aB`) atribuído diretamente a todas as entidades (`Croqui`, `Grupo`, `Setor`, `Escalada` e `PontoDeInteresse`).

Durante o processo de design, avaliou-se inicialmente uma abordagem intermediária com 3 tabelas YAML por croqui (mapeando UUIDs para inteiros de 4 bytes compactos). Porém, uma **análise quantitativa empírica sobre o acervo real** revelou que essa complexidade era injustificada:

- O peso real do acervo:** Os 42 croquis do país (637 setores, 4.318 vias, 4.811 POIs) somam **apenas 1,16 MB brutos (475 KB comprimidos)** de dados Protobuf, enquanto as fotos WebP somam **353 MB (99,7% do total)**. A diferença entre 8 bytes e 14 bytes por via representa meros kilobytes em todo o Brasil.
- O custo da complexidade:** Gerenciar 3 arquivos YAML por croqui exigiria criar ~130 arquivos de mapeamento adicionais, ferramentas de resolução de conflitos de merge (split de visões) no Git e pipelines complexos de tradução biunívoca.
- A vitória da Simplicidade (Regra VI do AGENTS.md):** O Pure NanoID 14c elimina 100% das tabelas intermediárias, zera os conflitos de merge de IDs no Git e resolve nativamente a integração física e digital.

2. As Armadilhas (Pitfalls) das Abordagens Alternativas

Pitfall 1: O Modelo Histórico de Referências por Nomes Textuais

A abordagem legada: Referenciar vias em mapas por cadeias de nomes: `(setor: "Gruta", escalada: "Sombra Fresca")`.

Por que falha: Renomear uma via quebra mapas em cascata, forçando varreduras custosas no editor e no Git. A resolução exige joins textuais lentos no app móvel. A eliminação das strings legadas no `database/` transforma renomeações em alterações atômicas de 1 linha.

Pitfall 2: Tabelas YAML Intermediárias e Split de Visões no Git

A abordagem intermediária: Criar arquivos `ids_entidades.yaml` e `ids_pontos.yaml` para mapear UUIDs para inteiros.

Por que falha: Branches concorrentes geram conflitos de merge constantes nos arquivos YAML centrais, exigindo parsers especiais para resolver marcadores `<<<<<< HEAD`. Criar e manter centenas de arquivos YAML no repositório introduz sobrecarga operacional desnecessária.

Pitfall 3: NanoID Curto demais (8 ou 10 caracteres)

A abordagem perigosa: Usar NanoIDs de 8 ou 10 caracteres para economizar caracteres em QR codes.

Por que falha: Pelo paradoxo do aniversário, com 10 caracteres Base62 (62^{10} ≈ $8,39 \times 10^{17}$), 1 milhão de vias acumuladas em forks mundiais gerariam probabilidade de colisão mensurável de $1 \text{ em } 1,6 \times 10^6$. Com **14 caracteres** (62^{14} ≈ $1,24 \times 10^{25}$), a probabilidade para 1 milhão de vias cai para $4 \text{ em } 10^{14}$ (1 em 25 trilhões — zero prático absoluto).

Pitfall 4: URLs Longas nas Placas Físicas de Aço Inox

A abordagem ingênua: Utilizar URLs como `https://app.arestaclimb.com/croqui/br_mg_cipoboulder/setor/via/x8siJek3FiG3aB`.

Por que falha: URLs com mais de 50 caracteres forçam QR Codes de Versão 5 ou superior (> 37x37 módulos), resultando em micropontos minúsculos que não suportam gravação a laser limpa em aço inox escovado, além de falharem rapidamente sob sol intenso ou poeira.

Pitfall 5: QR Codes com Correção Baixa (Nível L ou M) na Montanha

A abordagem ingênua: Priorizar menor tamanho de QR Code adotando correção de erro Nível L (7%) ou M (15%).

Por que falha: Em ambiente de montanha, uma placa exposta a 30 anos sofre arranhões de mosquetão, pó de magnésio (carbonato de magnésio aderido aos relevos da gravação), fezes de pássaros e oxidação superficial. O Nível L ou M falha catastrófica e precocemente. O Nível H recupera até 30% da área do código destruída.

3. A Solução: Arquitetura Pure NanoID 14c Universal

Toda entidade no repositório recebe nativamente um `uid` de 14 caracteres alfanuméricos contínuos em Base62. Não existem arquivos de tradução YAML.



4. Física e Engenharia das Placas nas Falésias (Aço Inox & QR Code)

O domínio curto `aresta.cc` reduz o prefixo da URL para apenas `https://aresta.cc/` (18 caracteres). Somado aos 14 caracteres do NanoID, o link canônico totaliza **exatamente 32 caracteres**.

Parâmetro	Valor Especificado	Fundamentação Técnica e Tolerância Física
URL Canônica	<code>https://aresta.cc/<uid></code>	Exatamente 32 caracteres (ex: <code>https://aresta.cc/x8siJek3FiG3aB</code>). Mínimo overhead em texto e QR code.
Versão do QR Code	Versão 3 (29 × 29 módulos)	Módulos amplos e bem espaçados em placas de 50mm × 30mm. Excelente nitidez em fibra ótica a laser.
Nível de Correção	Nível H (High — 30%)	Permite que até 30% da superfície do código seja danificada por riscos mecânicos ou sujeira sem perder leitura.
Capacidade V3 / H	35 caracteres (binário 8-bit)	A URL de 32 caracteres consome 32 dos 35 bytes disponíveis, mantendo folga de 3 caracteres dentro do envelope da Versão 3.
Material Físico	Aço Inoxidável AISI 316L	Resistência marítima e atmosférica a intempéries por mais de três décadas em rocha calcária, granito e quartzito.

5. Desacoplamento Relacional e Adoção do Termo Rótulo

1. Desacoplamento Relacional Total nos Mapas

Os campos textuais `escalada`, `setor` e `grupo` são **completamente deletados** dos arquivos Markdown de mapas na pasta `database/`. As referências passam a apontar unicamente para o `alvo_uid` e a lista ordenada `pontos_uids`. Renomear uma via torna-se uma operação trivial de 1 linha no atributo `nome` da via. O comando do editor `CmdRenomearEscalada` fica isento de varrer ou alterar mapas.

2. Termo rótulo em Português Brasileiro (Inegociável — AGENTS.md)

Em estrito cumprimento ao **Princípio I do AGENTS.md** ("Tudo em Português"), o campo de exibição visual em `PontoDeInteresse` é renomeado de `label` para `rotulo`. No Protobuf, `label` é marcado como `[deprecated = true]` e o compilador popula ambos os campos no binário para garantir compatibilidade com versões anteriores do aplicativo.

6. Governança de Migrações: Isolamento do Serving de Produção

Para evitar que migrações internas de banco de dados forcem a invalidação dos dados locais de usuários em falésias remotas, o arquivo `migracoes/0005_migrar_uids_e_rotulos.py` declara expressamente:

```
# Em migracoes/0005_migrar_uids_e_rotulos.py:
AFETA_VERSAO_SERVING: bool = False
```

O utilitário `serving/update_serving.py` inspeciona esta constante e **não incrementa a versão pública de serving** (mantendo `kDataVersion = 4`). O motor de migração atualiza os arquivos locais e marca `ultima_migracao: 5` nos croquis.

7. Testes de Contrato Arquiteturais e Biblioteca Pura

Seguindo o **Imperativo de Biblioteca em Primeiro Lugar (Library-First)** e **TDD com 100% de Cobertura**:

1. **Biblioteca Pura** `scripts/gerenciar_uids_lib.py`: Módulo independente com funções puras `gerar_uid()`, `validar_uid(uid)`, `formatar_url_aresta(uid)` e `extrair_uid_de_url(url)`. Testada com 100% de cobertura em `scripts/gerenciar_uids_lib_test.py`.
2. **Contrato da Pasta** `database/` (`tests/contrato_database_uids_test.py`): Audita todo o acervo. Falha se qualquer arquivo contiver referências com `escalada:`, `setor:` ou `grupo:`, se contiver `label:`, se faltar `uid:` de 14 caracteres, ou se existirem arquivos residuais `ids_*.yaml`.
3. **Contrato de Serialização do Editor** (`tests/contrato_editor_serializacao_test.py`): Garante que a serialização do editor nunca regrida emitindo campos depreciados.

8. Quadro Comparativo: Modelo Anterior vs. Pure NanoID 14c

Critério	Modelo Inicial (Tabelas YAML + IDs Inteiros)	Modelo Definitivo (Pure NanoID 14c Universal)
Complexidade Estrutural	3 tabelas YAML por croqui (~130 arquivos extras no Git)	Zero tabelas extras. UUIDs embutidos diretamente nos arquivos fonte.
Conflitos de Merge no Git	Requer algoritmo de split de visões para resolver conflitos de IDs	Risco zero de conflito de IDs (1_{24} imes 10^{25} \$ combinações).
Economia Real de Bytes	8 bytes vs 14 bytes por via (economia de ~25 KB em todo o país)	Impacto imperceptível (o acervo total de PB é 1,16 MB vs 353 MB de fotos).
Placas Físicas na Rocha	<code>app.arestaclimb.com/12/35</code> (26 caracteres)	<code>https://aresta.cc/x8siJek3FiG3aB</code> (32 caracteres, cabe em QR V3 Nível H).
Sincronização Offline-First	Requer mapeamento bidirecional complexo entre inteiros e nuvem	Perfeito para PowerSync e SQLite (chaves primárias de texto locais).
Aderência aos Princípios	Alta complexidade e camada de abstração pesada	Máxima Simplicidade e Anti-Abstração (Regra VI do AGENTS.md).

9. FAQ Técnico para Desenvolvedores e Mantenedores

- P: O que é o NanoID 14c Base62?**
R: É uma cadeia aleatória de 14 caracteres alfanuméricos contínuos sem hífen (`[0-9a-zA-Z]`), gerada criptograficamente via `secrets.choice` . Possui 62^{14} $\approx 1_{24}$ imes 10^{25} \$ combinações.
- P: Por que o QR Code Versão 3 com Correção Nível H é tão importante?**
R: Placas em falésias ficam expostas a sol, chuva, pó de magnésio e atrito de mosquetões. O Nível H tolera até 30% de destruição da área útil do QR Code mantendo a capacidade de leitura por smartphones.
- P: O que acontece se o usuário escanear a placa sem ter o app instalado ou sem internet?**
R: A URL `https://aresta.cc/<uid>` direciona para um worker leve na nuvem que redireciona para a via na web ou abre o aplicativo nativo via Universal Links / App Links.
- P: Como garantimos retrocompatibilidade com versões antigas do app móvel?**
R: O compilador `deploy_generated.py` popula em memória os campos legados (`escalaLada` , `setor` , `grupo` e `Label`) no `compilado.binarypb` , mantendo clientes anteriores funcionando perfeitamente em `v4` .